

14_founder_top1

Section 12: Founder Round

This section is the one where the resume and the architecture stop mattering and a single question takes over: *would I bet a company on this?* A founder is not grading my dbt DAG; they are pressure-testing whether I understand who pays, why, what kills the thing, and whether I can think in dollars and not just in p-values. I answer in the same voice I built Helios in — quantify, name the failure mode, state the alternative, be honest about the risk.

Business

Who actually pays for this, and why?

Ideal answer. The buyer is a growth or analytics leader at a company doing enough e-commerce or product volume that a one-point conversion swing is a real number — say a Head of Growth, VP Analytics, or a founder still wearing the data hat at a Series A–C company. They pay because root-cause analysis is the single most expensive recurring task on an analyst's plate: in my experience and in the literature it runs roughly **one to three analyst-days per anomaly**, and most teams see several genuine anomalies a month. The thing they are buying is not "insights" — dashboards already flood them with insights. They are buying *collapsed time-to-answer* and *defensible answers*: Helios turns a multi-day manual dig into a sub-five-minute autonomous run that arrives with a decomposition, a significance test, and a dollar number. **Why Helios demonstrates it:** the product's heartbeat is the scheduled autonomous run, not a chat box — the value proposition is literally "the analysis is already done when you open it."

What would you charge, and on what model?

Ideal answer. I'd anchor to the value, not the compute. One avoided misdiagnosis — telling a team to "fix checkout" when the real issue was acquisition mix — burns weeks of engineering and a quarter of misallocated spend, easily five figures. So I'd price as a per-tenant SaaS seat tied to data volume and run frequency, something like **2k–8k/month** for a mid-market growth team, with the framing "less than a fifth of one analyst's loaded cost, and it never sleeps." The honest constraint is my marginal cost: every run consumes BigQuery bytes and Opus/Sonnet tokens, so I'd gate runs under the fixed **5 GiB byte budget** per run and meter token spend, which is exactly why the architecture splits Opus (Orchestrator, Diagnose, Critic) from Sonnet (the high-volume

worker agents). **Why Helios demonstrates it:** the `dry_run` -before- `run_query` byte gate isn't just a correctness control, it's the unit-economics control that makes a per-run price sane.

How big is the market — TAM?

Ideal answer. I'd size it bottom-up, not with a top-down "digital analytics is a \$X0B market" hand-wave that founders rightly distrust. The serviceable wedge is companies that already run GA4 (or a warehouse-native equivalent) on BigQuery and employ at least one analyst — that's hundreds of thousands of mid-market and enterprise e-commerce and product orgs globally. If even a fraction carry an analytics tooling budget of a few thousand a month, the reachable market is comfortably in the **low billions of ARR**. But I'd be candid that TAM is a fundraising slide; what matters at the seed stage is whether *ten* design-partner teams will pay, and I'd rather over-index on that proof than on a big number.

Growth

What's the wedge, and what's the moat?

Ideal answer. The **wedge** is the single most painful, most repeated, least-automated workflow: "conversion dropped — why?" It's narrow, urgent, and lands inside a team that already has the data. I don't try to replace the BI stack; I sit on top of it and answer the one question dashboards can't. The **moat** is *governed trust*, and it has three reinforcing layers that compound over time: (1) the semantic layer means SQL is composed from a governed registry, so there are **zero hallucinated columns** — a generic LLM-on-your-warehouse competitor cannot promise that; (2) the **offline labeled benchmark** (85% root-cause accuracy vs 45% naive baseline, gated in CI) is a defensible, auditable trust claim that takes real effort to build; and (3) the **memory store** — diagnosis history, suppression list, action-tracking — gets more valuable per tenant the longer Helios runs, because it learns that tenant's seasonality and which fixes actually worked. **Why Helios demonstrates it:** the three defensibility pillars (governed-SQL grounding, deterministic stats, adversarial Critic + eval) are not just engineering hygiene — they are the competitive wall.

Build vs. buy vs. just use the incumbent — why doesn't GA4/Looker/an in-house script win?

Ideal answer. The incumbents solve a *different* problem and that's the opening. GA4 and Looker are reporting surfaces — they tell you *what* happened with great fidelity and say nothing about *why*; the analyst still does the root-cause dig by hand. An in-house "ask-your-data" LLM script is the dangerous middle option: it's cheap to start and silently wrong, because it hallucinates columns and computes statistics in token-space. My whole architecture is a structural answer to that failure: the LLM **never authors SQL** (semantic-mcp is the only path) and **never computes a statistic** (stats-mcp is the only path). So the build-vs-buy pitch is "you could wire GPT to your warehouse in a weekend, and it would confidently misdiagnose your funnel — the hard part isn't

generating an answer, it's making it *correct and trusted*, and that's a quarter of engineering you don't want to own." **Why Helios demonstrates it:** the five-MCP trust boundary is precisely the part a customer cannot cheaply rebuild.

ROI

What is the ROI of a single Decision Brief?

Ideal answer. I'd make it concrete with the canonical story. Aggregate `session_conversion_rate` falls week-over-week; the obvious read is "checkout is broken," and a team acts on it — eng sprints on the checkout flow, PM reprioritizes. Helios runs `decompose_change` and shows the per-segment rates are *flat*: the drop is almost entirely **mix** — a surge of low-converting Paid Social mobile traffic shifted the weights. The prescription flips from "fix checkout" (weeks of wasted eng, zero lift) to "fix acquisition mix." The ROI of that one brief is the **misdiagnosis cost avoided**: a wasted eng sprint plus a quarter of mis-aimed ad spend, against a brief that costs a few dollars of BigQuery and tokens. Even one such save a quarter pays for the product many times over. And the brief carries a **dollar revenue-at-risk** number natively, so the buyer doesn't have to construct the ROI — Helios hands it to them. **Why Helios demonstrates it:** revenue-at-risk ($\approx \Delta \text{rate} \times \text{affected sessions} \times \text{downstream value}$) is a first-class field on every finding, not an afterthought.

How does the value change as the company scales?

Ideal answer. It compounds in three directions. First, **data volume**: more segments and more traffic mean more places for Simpson's paradox to hide, so the mix-vs-rate decomposition gets *more* valuable, not less — exactly when manual RCA gets harder. Second, **organizational distance**: at scale the person who can diagnose a funnel and the person who owns the budget are different people in different time zones; the autonomous scheduled brief bridges that gap without a meeting. Third, **memory**: the action-tracking loop ("did the fix work?") and seasonality calendars mean the system's accuracy and relevance climb with tenure on the account, which lowers churn. The honest counter-pressure is cost — more data is more bytes — which is why warehouse-agnostic adapters and per-tenant byte budgets are on the roadmap rather than pretending scale is free. **Why Helios demonstrates it:** the memory store and the deterministic decomposition are the two components whose value is explicitly super-linear in scale.

Product Strategy

What would you build next?

Ideal answer. I sequence by trust-then-reach. The immediate next build is **closing the action-tracking loop** — automatically reading back whether a prescribed experiment shipped and

whether the fix moved the metric, via quasi-experimental pre/post and difference-in-differences. That turns Helios from "diagnoser" into "accountable diagnoser," which is the retention story. After that, **warehouse-agnostic adapters** behind warehouse-mcp (Snowflake, Databricks, DuckDB) so the semantic layer stays the only SQL author regardless of dialect — that's the reach unlock beyond GA4-on-BigQuery. I would *not* prematurely build a chat UI or a dashboard; that's drifting into the anti-product. **Why Helios demonstrates it:** the roadmap is already staged P0→P4 with the principle "never start a phase until the prior phase's exit criteria are green in CI," which is the same trust-first discipline.

How do you know customers actually want this?

Ideal answer. I'd be honest that I can't fully prove demand from a portfolio project on a public dataset — that's a limitation, not a strength to oversell. What I *can* point to is the strength of the underlying pain signal: every analytics team I've worked with or studied spends real days on manual RCA and routinely confuses mix-shift with rate-change. The validation plan is design partners: get five-to-ten growth teams to run Helios on their own GA4 export for a month and measure whether the briefs change a real decision. The kill-criterion is explicit — if leaders read the brief and still send it to an analyst to "double-check," I haven't earned trust and the value prop fails. **Why Helios demonstrates it:** the offline benchmark is my honest proxy for "is it right" before I have real users; demand validation is the next experiment, designed the same disciplined way.

What kills this product?

Ideal answer. Three things, in order of seriousness. (1) **Trust collapse** — if Helios confidently ships one wrong root cause to an executive, the relationship is over; that single risk is why the entire architecture is built around verify-then-trust, the adversarial Critic, and the 85%/45% benchmark gate. (2) **The "good-enough" incumbent** — if GA4 or Looker bolts on a credible native explanation feature, my wedge narrows; my defense is depth (real decomposition, governed stats, the accountability loop) and warehouse-agnosticism. (3) **Unit economics** — if per-run BigQuery and token cost outpaces what a tenant will pay, the model breaks; the byte budget and the Opus/Sonnet split are the live mitigations, but it's a real constraint I watch. The mature read is that the existential risk is almost entirely #1 — and it's the one I've invested the most architecture in defending against. **Why Helios demonstrates it:** "the hard part is not generating insights — it is making them correct and trusted" is the thesis, so the thing that kills it is the thing the thesis is built to prevent.

Section 13: Top 1% Interview Prep

This is the section that separates a candidate who *built* the project from one who can *defend* it under adversarial questioning. The structure is deliberate: master a small set of concepts cold, know the boundary of what I can hand-wave, pre-arm the weak spots, disarm the traps, and walk in with a tight rehearsal plan.

Concepts you MUST understand deeply

These are non-negotiable. If I fumble any of them, the project's credibility falls apart.

- **Mix-vs-rate decomposition and Simpson's paradox.** I must be able to write $R = \sum w_i \cdot r_i$ and derive $\Delta R = \text{mix} + \text{rate} + \text{interaction}$ ($\sum \Delta w_i \cdot r_i(t_0) + \sum w_i(t_0) \cdot \Delta r_i + \sum \Delta w_i \cdot \Delta r_i$) on a whiteboard, and explain *why* drilling rate effects (real behavior) before mix effects (composition artifacts) dissolves Simpson's paradox. The clinching detail: rates must be computed as $\text{SUM}(\text{num})/\text{SUM}(\text{den})$ after grouping, never an average of per-segment ratios — that re-aggregation discipline *is* the defense.
- **Sessionization.** GA4 ships no session row; I reconstruct it from events sharing `(user_pseudo_id, ga_session_id)`, with the canonical hashed `session_key`. I must explain why this matters: get sessionization wrong and every downstream number is silently wrong, which is why it's a tested keystone.
- **The grounding boundary.** Why the LLM never authors SQL (semantic-mcp) and never computes a statistic (stats-mcp), and why that boundary is *structurally* enforced via MCP tool allow-lists rather than just asked-for-nicely in a prompt.
- **Experiment power and MDE.** I must define MDE, explain the power/ α /baseline-rate/sample-size relationship, and connect it to `experiment-mcp.power_analysis` and `runtime_estimate` — every prescribed card is powered, not guessed.
- **The eval methodology.** Offline labeled benchmark: inject known rate-only or mix-only perturbations into a frozen GA4 copy, make the pipeline rediscover them, grade root-cause accuracy and decomposition MAPE. I must explain *why offline* (the real data's anomalies are unlabeled) and what the 85%-vs-45% claim actually measures.
- **Why determinism matters.** All math runs as seeded Python in stats-mcp, not in token-space, because reproducibility and auditability are the trust contract — an executive decision cannot rest on a number an LLM might generate differently next run.

Concepts you can explain at a high level

Know enough to be credible and to say "I'd reach for X" — but don't oversell mastery I don't have.

- **Prophet internals.** Additive decomposition (trend + seasonality + holidays) with a piecewise-linear trend and Fourier-series seasonality; I use it via `stats-mcp.forecast` for

expected-vs-actual residuals. I can name the alternative (pmdarima/auto-ARIMA) and the tradeoff (Prophet is robust and low-tuning; ARIMA can be tighter on stationary series).

- **MCP wire protocol.** A JSON-RPC client-server protocol; stdio transport for the in-process tool servers, streamable-HTTP for warehouse-mcp; the value is schema-typed tool calls that make the grounding boundary physically enforceable. I don't need to recite the spec.
- **dbt incremental internals.** `insert_overwrite` partitioned by `event_date`, clustered by `device_category`, `channel_group`, which gives idempotent re-runs over a partition. I can explain *why* (cost + no duplicate diagnoses) without reciting every dbt macro.
- **Vector-store internals.** Embeddings of prior diagnoses for similarity recall in the memory store; I understand ANN/cosine-similarity retrieval conceptually. I won't pretend to have benchmarked HNSW parameters.

Likely weak spots

I name these *before* the interviewer does — it reads as maturity.

- **Live A/B vs. observational.** The dataset is observational and historical (Nov 2020–Jan 2021). Helios **designs and sizes** experiments and runs quasi-experimental readbacks (pre/post, difference-in-differences); it does **not** run live A/Bs. I say this proactively.
- **Causal inference.** The decomposition is *correlational* attribution, not causal identification. The honest framing: Helios separates composition from behavior and the Critic flags confounds, but true causal claims (DiD with parallel-trends checks, synthetic control, double-ML) are the P4 frontier, gated on data that supports them.
- **Cost at scale.** Per-run bytes and tokens are a real constraint; the byte budget and Opus/Sonnet split are mitigations, and warehouse-agnostic per-tenant budgets are roadmap, not done.
- **Cross-device identity.** `user_id` is almost always null, so identity is cookie-grain (`user_pseudo_id`); user-level metrics are explicitly cookie-based approximations and stitching is unsound on this data.
- **Statistical depth under probing.** If pushed past the surface (e.g., multiple-comparisons correction across many segments, choice of significance test for proportions, confidence intervals on decomposition terms), I show I know the *direction* of the right answer and where it lives in stats-mcp, rather than bluffing a derivation.

Interviewer traps

Pre-scripted disarms for the predictable provocations.

- **"So it's just a chatbot."** No — conversation is a *secondary* drill-down surface. The product is the autonomous scheduled run that produces a Decision Brief proactively. A chatbot is reactive and ungoverned; Helios is proactive, governed, and graded.

- **"85% isn't production-grade."** Correct, and the comparison is the point: it's 85% vs a **45%** naive baseline on a labeled benchmark, gated in CI so accuracy can't silently regress. The number is a floor that ratchets up, and the Critic + reconciliation catch the failures that slip past raw accuracy.
- **"Why not just pandas?"** Pandas in an agent's hands means the LLM writing transformation code — back to ungoverned generation and silent errors. stats-mcp wraps scipy/statsmodels/prophet behind a *typed, deterministic, seeded* tool surface so the math is auditable and the LLM can only *call* it, never author it.
- **"Show me the SQL the LLM wrote."** Trick question — the LLM never wrote SQL. `semantic-mcp.build_query` composes it from the governed registry; I can show the *governed* SQL and trace every column to the GA4 schema, which is exactly the zero-hallucination guarantee.
- **"You've mixed up mix and rate."** I slow down and re-derive: $\text{mix} = \sum \Delta w_i \cdot r_i(t_0)$ (composition moved, behavior fixed), $\text{rate} = \sum w_i(t_0) \cdot \Delta r_i$ (behavior moved, composition fixed). If I can re-derive it live, the trap becomes a strength.
- **"You said you run experiments."** I correct it immediately: I *design and size* experiments and run *quasi-experimental* readbacks; the data is observational, so claiming live A/Bs would be the dishonest answer.

Final preparation checklist

A tickable list tuned for placement season across Product/Growth/Data Analytics, Analytics Engineering, and AI/Data roles.

- ☐ **Rehearse the 60-second pitch and the 2-minute deep-dive** until they're muscle memory; lead with the autonomous-run + mix-vs-rate hook, not the tech stack.
- ☐ **Rehearse the four STAR stories** (the mix-shift insight; trust via the eval harness; the MCP tool-boundary; the `traffic_source` first-touch gotcha) — each lands a different competency.
- ☐ **Memorize from the Fact Sheet:** the seven funnel stages in order; the seven agents and their models (Opus: Orchestrator/Diagnose/Critic; Sonnet: Monitor/Decompose/Prescribe/Narrator); the five MCP servers and which is the *only* path to SQL vs math; the ten channel groups.
- ☐ **The one diagram to draw cold:** the seven-agent plan-execute-critique loop with the Critic as a gate, the five MCP servers as the trust boundary, the funnel, and the identity $\Delta R = \text{mix} + \text{rate} + \text{interaction}$.
- ☐ **The three numbers to never forget:** 85% vs 45% (root-cause accuracy vs naive baseline); 0 hallucinated columns; <5 min per run under a 5 GiB byte budget.
- ☐ **Role-tune the opener:** for Product/Growth lead with revenue-at-risk and the prescribed experiment backlog; for Analytics Engineering lead with the dbt DAG, the semantic registry,

and reconciliation; for AI/Data lead with the MCP grounding boundary, the agent FSM, and the eval gate.

- ☐ **Pre-arm every weak spot and trap above** so each becomes a maturity signal instead of a stumble.
- ☐ **Mock-interview targets:** at least three full mocks — one technical-deep (decomposition + sessionization + eval), one founder/commercial (Section 12 questions), one behavioral (the STAR stories) — with someone allowed to interrupt and push back.